SIMATS ENGINEERING

ASSESSMENT TOOL 1 : Concept Mapping

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO1: *Apply cloud computing technologies including hardware-software evolution, virtualization, web services, and service models to develop cloud-based solutions.* (BL2)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Imitation (Level 1) **Total Marks: 50**
Question Count: 4

Code of Conduct

I Thamaraiselvi S (192511042) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: Thamaraiselvi S

Assessment Tasks

Concept Mapping Tasks

Create concept maps for the following tasks. Each map must show key nodes, link labels, and a logical hierarchy.

1. Concept Map 1: Cloud Computing Fundamentals. Include definition, characteristics, benefits, and limitations.
2. Concept Map 2: Evolution of Cloud Computing. Connect hardware evolution, internet software evolution, distributed systems, and service delivery.
3. Concept Map 3: Service Models. Map IaaS, PaaS, SaaS, and XaaS with provider responsibilities and user responsibilities.
4. Concept Map 4: Virtualization and Web Services as Enablers of Cloud. Show how these two concepts support cloud deployment and scalability.

Expected concept nodes:

- Elasticity, scalability, resource pooling, on-demand access, measured service
- Virtual machine, hypervisor, abstraction, API, SOAP, REST, provider, consumer
- Infrastructure layer, platform layer, application layer, shared responsibility

Concept Mapping Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Identification	25%	All major concepts are accurately identified	Most concepts are identified correctly	Limited concepts are identified	Essential concepts are missing
Relationship Mapping	25%	Links between concepts are	Most links are correct	Some links are weak or unclear	Relationships are mostly incorrect

		accurate and meaningful			
Hierarchy and Organization	20%	Excellent structure with clear hierarchy	Mostly organized with minor issues	Basic structure with weak hierarchy	No logical organization
Technical Relevance	20%	Map strongly reflects cloud course content	Mostly aligned to topic	Partially aligned to topic	Weak alignment to topic
Visual Clarity	10%	Neat, readable, and easy to interpret	Generally readable	Somewhat cluttered	Poorly presented

Permitted Tools :

draw.io / diagrams.net, Lucidchart, MindMeister, Miro, Canva, PowerPoint, Google Drawings, or equivalent diagramming tool

Procedure for Preparing the Digital Concept Map

1. Use the following procedure to prepare the diagram and upload evidence.
2. Students who already know another diagramming tool may use it, but the exported evidence and source file must still be submitted.
3. Open https://app.diagrams.net/ or the desktop version of draw.io / diagrams.net.
4. Choose Device, Google Drive, or OneDrive as the save location according to availability.
5. Create a blank diagram and name the file as CSA15_CO1_AT1_RegisterNumber_Concept_Map.drawio.
6. Place the concept map title at the center or as a clearly visible anchor node.
7. Add major cloud nodes: cloud definition, cloud evolution, cloud architecture, service models, deployment models, virtualization, web services, SOAP, REST, APIs, elasticity, scalability, resource pooling, measured service, and emerging paradigms.
8. Connect nodes with labeled arrows such as uses, depends on, enables, scales through, communicates through, belongs to, supports, provides, consumes, or controls.
9. Use colors meaningfully: one color for service models, one for cloud characteristics, one for API/web-service concepts, and one for concept map title-specific elements.
10. Export the final diagram as PNG or PDF. Keep the editable .drawio source file also.
11. Paste the exported diagram image or screenshot in this DOCX under Task 1.
12. Upload the DOCX, exported PNG/PDF, and .drawio file to Google Classroom.

Task 1: Concept Map

Create a concept map in draw.io, Lucidchart, Miro, Figma, or any suitable diagramming tool. The map must connect cloud ecosystem, cloud actors, service models, deployment models, virtualization, web services, REST/SOAP APIs, scalability, elasticity, and the student's Project.

Task 2A: Service Model Responsibility Matrix

Prepare a technical matrix that compares SaaS, PaaS, IaaS, FaaS/serverless, database-as-a-service, storage-as-a-service, identity-as-a-service, and monitoring/logging services. For each selected service category, explain what is managed by the cloud provider, what is managed by the student application team, and why that split fits the Project.

Task 2B: Cloud Service Decision Matrix

Map Project modules to cloud services. Include authentication, frontend hosting, backend/API layer, database, object storage, analytics/logging, notification, AI/Python module, security, CI/CD, backup, and monitoring. Students should justify the most relevant service choices instead of forcing every possible cloud service into the design.

Task 3: API Flow, Elasticity, and Scalability Design

Draw a professional cloud-flow diagram for the Project modules. The diagram must show user app or web app, API gateway/backend service, authentication, compute layer, database, object storage, analytics/logging, monitoring, security boundaries, and scaling points. Students may replicate the given visual pattern, but all components and labels must be customized to their own architecture.

Task 4: Technical Justification

Write a concise justification explaining how the diagram and matrices prove CO1. The justification must discuss service model selection, API role, elasticity, scalability, virtualization or managed compute choice, and cloud responsibility boundaries.

1. Evaluate the effectiveness of your concept map in representing cloud computing principles. Which design decisions most strongly support your solution, and why?
2. Analyze how scalability, elasticity, and resource management influence the architecture shown in your concept map. What challenges could arise if these capabilities were absent?
3. Justify the selection of the APIs, web services, or cloud services included in your concept map. How do they contribute to the overall functionality and efficiency of the system?
4. Critically examine the relationships between the components in your concept map. How do these interactions improve reliability, performance, or user experience?
5. Reflect on the process of developing your concept map. How has creating and analyzing the map changed your understanding of cloud computing architecture, and what improvements would you make if you were redesigning it?

Expected Digital Evidence

- Editable diagram source file and exported PNG/PDF for the concept map.
- Completed service model responsibility matrix and cloud service decision matrix.
- Architecture/API flow diagram exported as PNG/PDF.
- README file containing title, tools used, repository structure, and a short CO1 reflection.

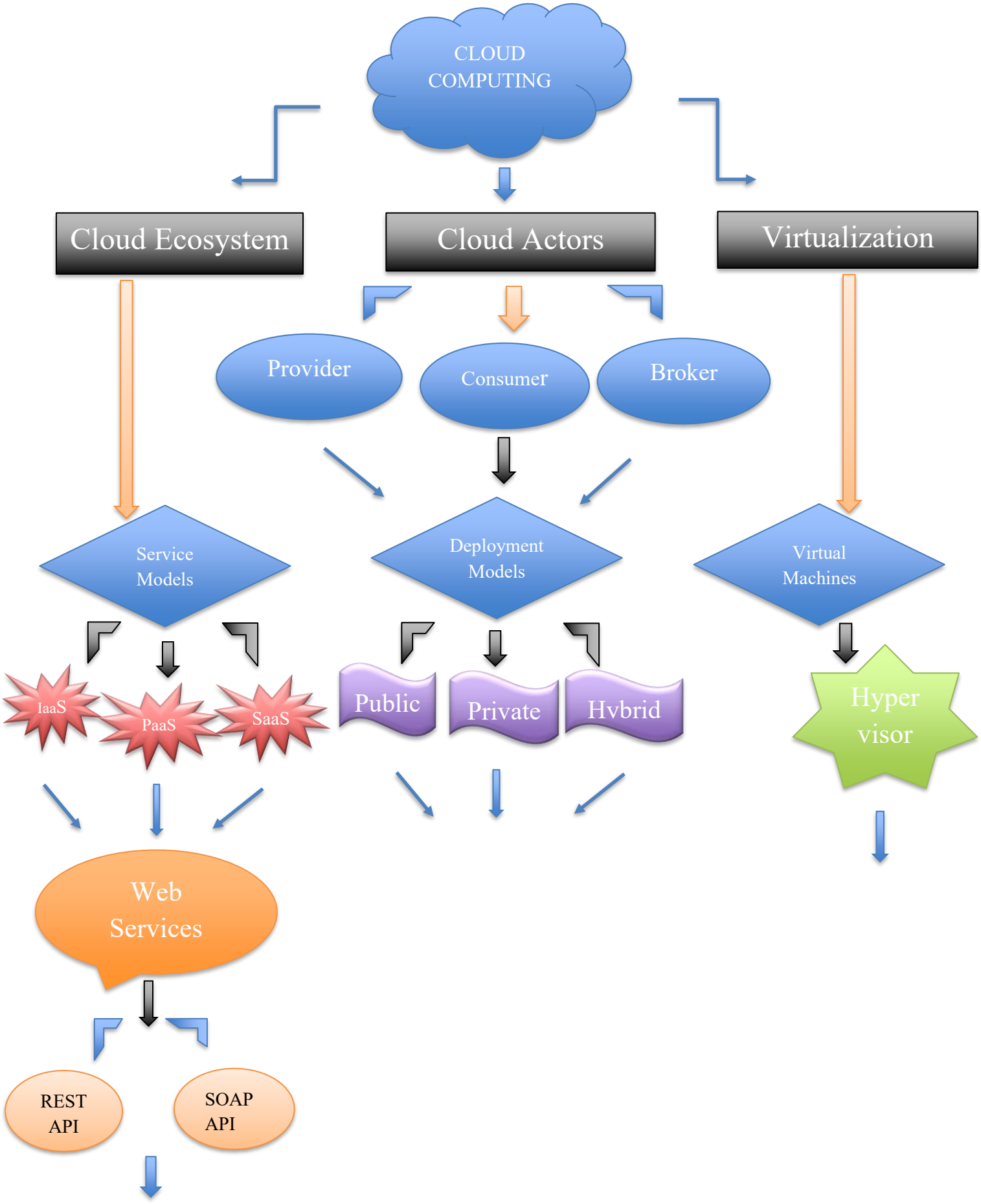
GitHub and Google Classroom Submission

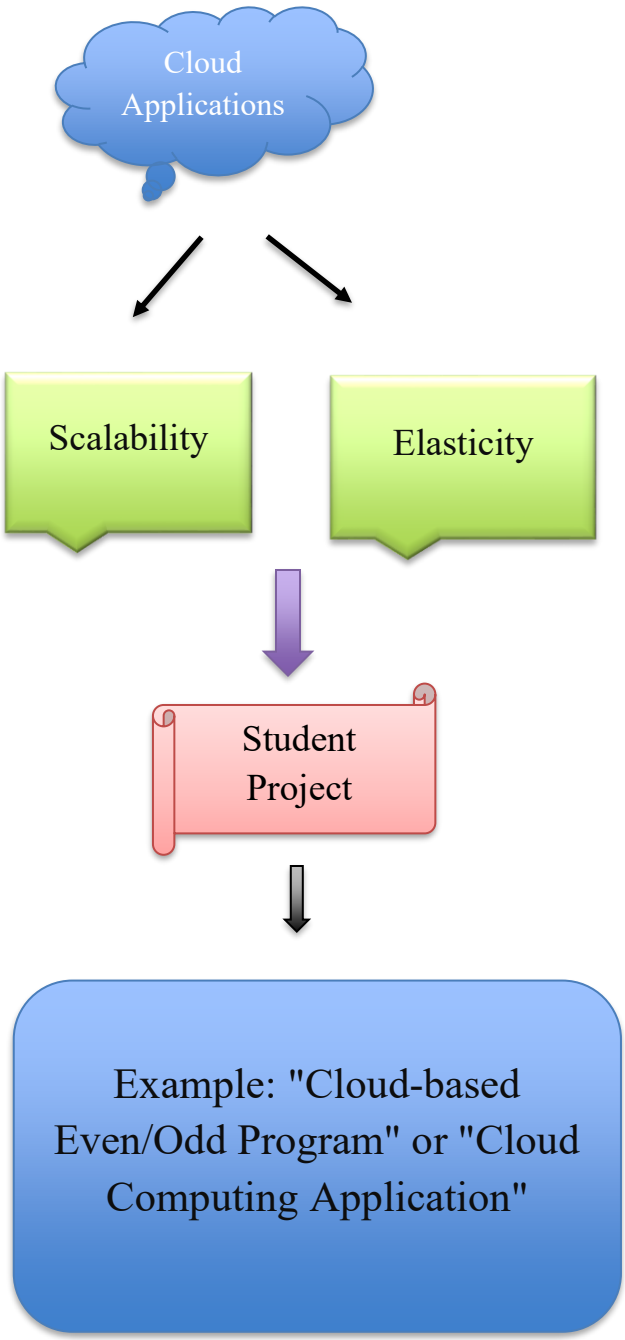
1. Create a repository using a clear name: CSA15_CO1_AT<Number>_<RegisterNumber>.
2. Upload editable source files, exported PDF/PNG evidence, screenshots, and a short README explaining the work.
3. Keep filenames professional and include the register number where appropriate.
4. Submit only the GitHub repository link in Google Classroom before the deadline.
5. Ensure the repository is accessible to the faculty evaluator during assessment.

Implementation Note

Faculty may evaluate the submitted evidence digitally using repository inspection, diagram quality, oral explanation, and CO-aligned rubric scoring. The focus is on technical understanding, cloud design reasoning, and hands-on evidence rather than routine written work.

Task 1: Concept Map




Cloud Applications
Scalability
Elasticity
Student Project
Example: "Cloud-based Even/Odd Program" or "Cloud Computing Application"

Task 2A: Service Model Responsibility Matrix

Service Category	Managed by Cloud Provider	Managed by Student Application Team	Why it Fits the Project
SaaS (Software as a Service)	Software application, updates, security, infrastructure	User accounts, application usage, data entry	Students can use ready-made cloud applications without maintaining software.
PaaS (Platform as a Service)	Operating system, runtime, middleware, servers, networking	Application code, testing, deployment	Allows students to focus on coding instead of server management.
IaaS (Infrastructure as a Service)	Physical servers, storage, networking, virtualization	Virtual machines, operating system configuration, applications, security settings	Suitable when students need complete control over the computing environment.
FaaS / Serverless	Infrastructure, server management, scaling, runtime	Individual functions, application logic, event triggers	Ideal for executing small functions without managing servers.
Database as a Service (DBaaS)	Database installation, backups, updates, availability, scaling	Database schema, tables, queries, application data	Simplifies data storage and management for student projects.
Storage as a Service (STaaS)	Storage hardware, replication, backup, availability	Uploading, organizing, retrieving, and deleting files	Useful for storing project files, documents, and datasets securely.
Identity as a Service (IDaaS)	Authentication, authorization, user identity management, security	User roles, access permissions within the application	Provides secure login and access control without building authentication from scratch.

Service Category	Managed by Cloud Provider	Managed by Student Application Team	Why it Fits the Project
Monitoring & Logging Service	System monitoring tools, log collection, alerts, dashboards	Application logs, performance analysis, error handling	Helps students identify issues, monitor performance, and improve application reliability.

Conclusion

This responsibility matrix clearly defines the roles of the **cloud provider** and the **student application team**. The cloud provider manages the underlying infrastructure and platform services, while the student team focuses on developing, testing, and maintaining the application. This division reduces operational complexity and allows the project to be developed more efficiently using cloud computing services.

Task 2B: Cloud Service Decision Matrix

Project: Cloud-based Even/Odd Program (Python)

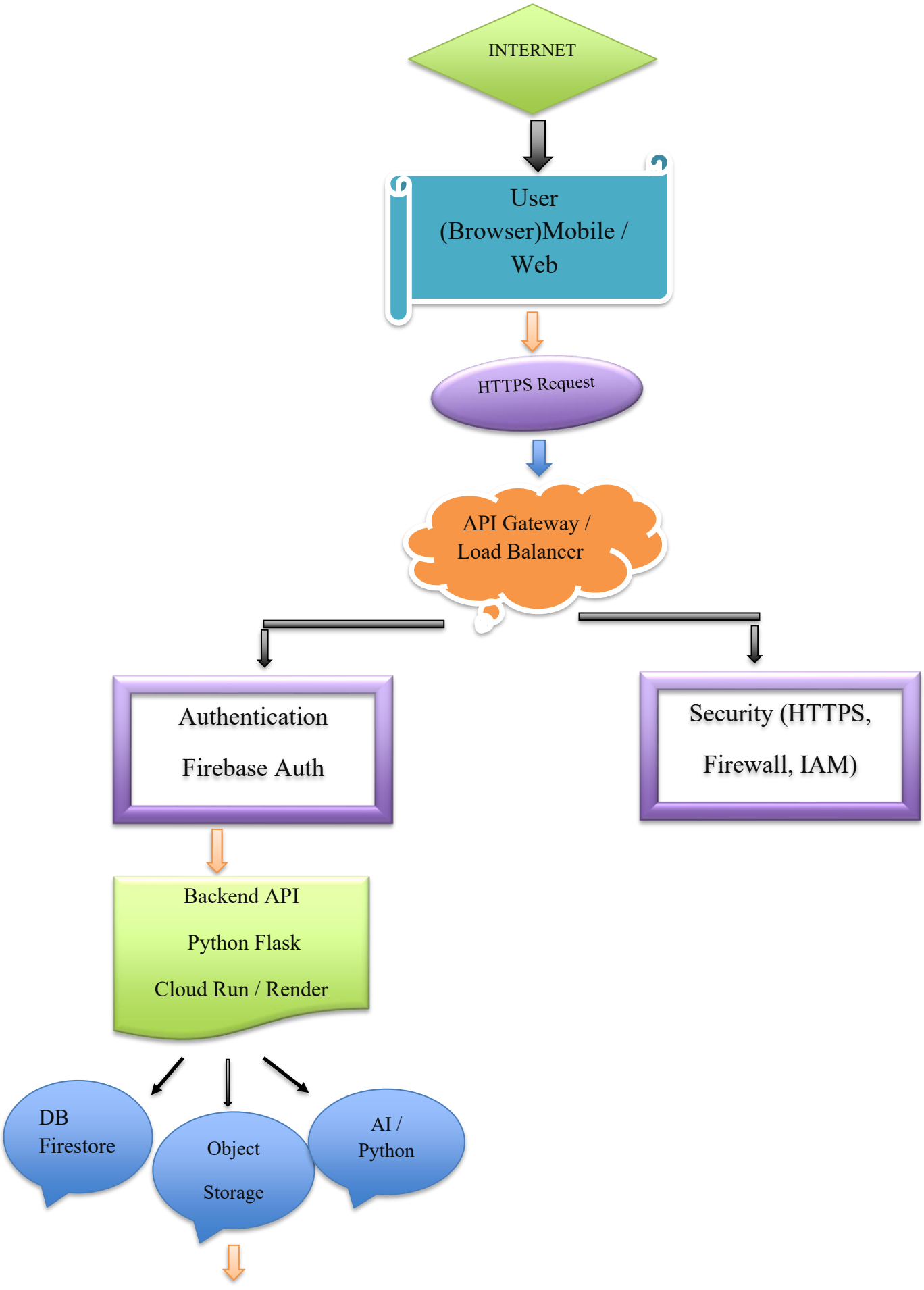
Project Module	Recommended Cloud Service	Purpose	Justification
Authentication	Firebase Authentication	User login and authentication	Easy to implement and provides secure authentication.
Frontend Hosting	GitHub Pages / Firebase Hosting	Hosts the web interface	Fast, reliable, and suitable for small student projects.
Backend / API Layer	Python Flask API (Google Cloud Run or Render)	Executes application logic	Supports Python applications and scales automatically.
Database	Firebase Firestore	Stores user details and application data	NoSQL database with simple integration and real-time updates.
Object Storage	Firebase Storage	Stores files, images, and project documents	Secure cloud storage with easy access.
Analytics / Logging	Google Cloud Logging	Records application logs and errors	Helps identify bugs and monitor application performance.
Notification	Firebase Cloud Messaging (FCM)	Sends notifications to users	Easy cloud-based notification service.
AI / Python Module	Google Colab / Vertex AI	Runs Python code or AI models	Supports Python programming and machine learning experiments.

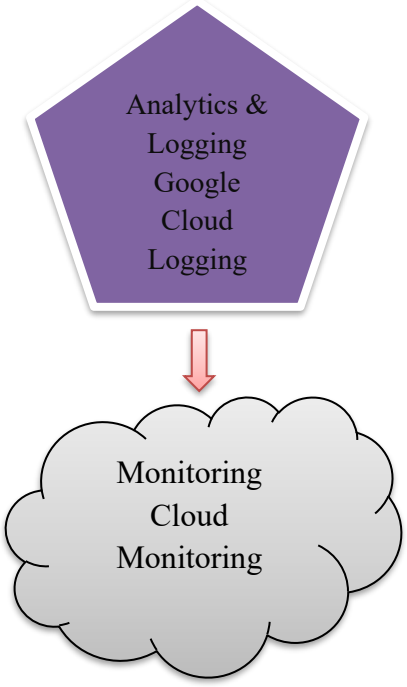
Project Module	Recommended Cloud Service	Purpose	Justification
Security	Firebase Security Rules & HTTPS	Protects application and user data	Provides secure communication and controlled access.
CI/CD	GitHub Actions	Automates testing and deployment	Automatically deploys updates whenever code changes are pushed.
Backup	Firebase Backup / Cloud Storage Backup	Stores backup copies of application data	Prevents data loss and enables recovery.
Monitoring	Google Cloud Monitoring	Tracks application health and performance	Detects issues and monitors system availability.

Justification

The selected cloud services are simple, reliable, and well suited for a student cloud computing project. Firebase provides authentication, database, storage, notifications, and security in one platform, reducing development effort. Google Cloud services support monitoring, logging, and Python execution, while GitHub Actions automates deployment. These services cover all essential project requirements without adding unnecessary complexity.

Task 3: API Flow, Elasticity, and Scalability Design




Analytics &
Logging
Google
Cloud
Logging
Monitoring
Cloud
Monitoring

ELASTICITY & SCALABILITY POINTS:

API Gateway → Automatically distributes incoming requests.

• Backend API → Auto-scales by creating multiple instances.

• Firestore Database → Automatically scales storage and queries.

• Object Storage → Unlimited scalable storage.

• Cloud Monitoring → Tracks CPU, memory, latency, and traffic.

• Security Boundary → HTTPS + IAM + Firewall protects all services.

Components Used:

Component	Purpose
User (Web/Mobile)	Accesses the application.
API Gateway / Load Balancer	Routes requests and balances traffic.
Authentication	Verifies user identity using Firebase Authentication.
Security Boundary	Protects communication with HTTPS, IAM, and firewall rules.
Backend API	Executes Python/Flask application logic.
Database	Stores application data in Firestore.
Object Storage	Stores uploaded files and documents.
AI/Python Module	Runs Python or AI-related processing.
Analytics & Logging	Records application logs and usage.

Component	Purpose
Monitoring	Monitors health, CPU, memory, and performance.
Scalability	Automatically adds more backend instances during high traffic.
Elasticity	Increases or decreases cloud resources based on demand.

Task 4: Technical Justification

The concept map, responsibility matrix, service decision matrix, and cloud architecture diagram collectively demonstrate **CO1 (Understand Cloud Computing Fundamentals and Cloud Service Models)**. The project uses a combination of **SaaS, PaaS, and managed cloud services** to simplify development while ensuring reliability and security. **Firebase Authentication** is selected for secure user access, **Firestore** for database management, **Firebase Storage** for file storage, and a **Python Flask backend** deployed on managed compute (such as Google Cloud Run) to process application requests.

The **API layer** acts as the communication bridge between the user interface and backend services. It receives requests from the web application, validates users through authentication, executes business logic, accesses the database or object storage, and returns the appropriate response. This modular API-based architecture improves maintainability and enables future integration with additional services.

The design incorporates **elasticity** by automatically increasing or decreasing compute resources according to user demand. **Scalability** is achieved through load balancing, auto-scaling backend services, and cloud-managed databases that can handle increased workloads without affecting application performance. These features ensure the application remains responsive as usage grows.

Instead of managing virtual machines directly, the project uses **managed compute services (PaaS/Serverless)**. The cloud provider manages infrastructure, operating systems, runtime environments, networking, and automatic scaling, while the student application team focuses on developing the application, API logic, database structure, and user interface. This **shared responsibility model** clearly defines the responsibilities of both the cloud provider and the application team, making the project secure, efficient, scalable, and aligned with modern cloud computing practices.